

Exhaustive Engineering Guide & Sprint Plan

PRODUCT NAME	Staye
DOCUMENT TITLE	Backend Infrastructure: Supabase to AWS Migration Runbook
REVISION	Revision 3 (V3) — Exhaustive Step-by-Step UI Guide
TARGET ARCHITECTURE	Phase 1 (EC2, RDS PostgreSQL, Cognito, S3)
TARGET AUDIENCE	Backend Engineering & DevOps Team
PREPARED BY	Zenxvio Engineering Team
DATE	July 2026

Purpose of This Document

This runbook provides an **exhaustive, highly-detailed manual** for the engineering team to migrate the Staye platform's backend infrastructure off Supabase and onto a production-grade AWS environment.

Unlike a standard high-level overview, this V3 document specifies every single UI click, configuration dropdown, and terminal command required to successfully provision the AWS environment, refactor the Next.js codebase, containerize the application, and migrate live data.

Table of Contents

- 1. Migration Architecture & Topology Overview
 - 2. Sprint 1: Cloud Foundation & IAM Security
 - 3. Sprint 2: Network Security & Firewalls
 - 4. Sprint 3: Exhaustive AWS Resource Provisioning
 - 5. Sprint 4: Codebase Refactoring (Moving off Supabase)
 - 6. Sprint 5: Dockerization & Automated CI/CD Pipeline
 - 7. Sprint 6: Data Cutover & Go-Live
-

1. Migration Architecture & Topology Overview

1.1 What Stays & What Changes

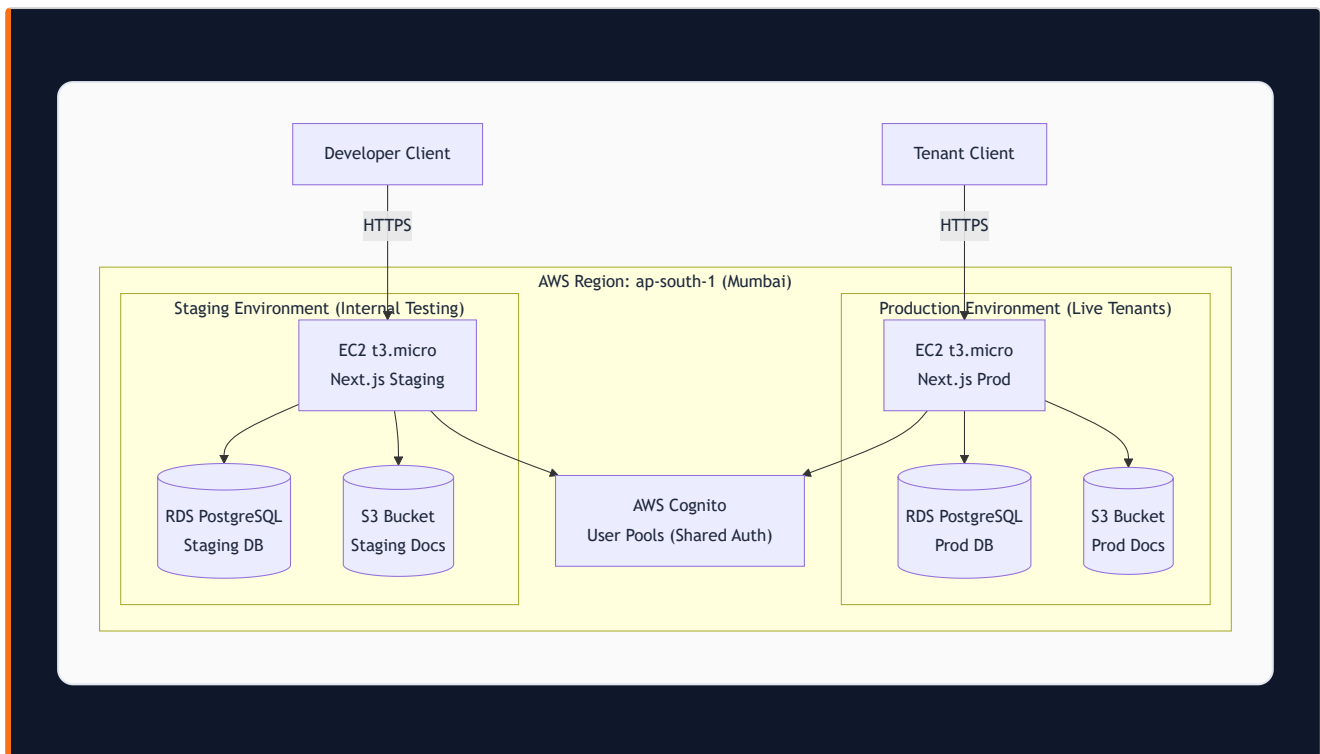
Staye is built on **Next.js (App Router)** utilizing the **Prisma ORM**. This provides a massive advantage during migration, as the core business logic remains entirely intact.

- **What DOES NOT change:** Database schemas, Prisma queries (`prisma.tenant.findUnique()`), backend business logic (rent calculations, hostel scoring), and frontend UI components.
- **What MUST be rewritten:**
 1. Authentication logic (migrating from `supabase.auth` to AWS Cognito).
 2. File upload logic (migrating from `supabase.storage` to `@aws-sdk/client-s3`).
 3. Environment variables, CI/CD deployment pipelines, and secrets management.

1.2 Target Topology: Dual Environment (Staging & Prod)

To ensure developers never test experimental code on live tenant data, we are separating Staging and Production at the infrastructure level.

⚠️ CRITICAL AWS FREE TIER WARNING ⚠️ The AWS Free Tier mathematically covers exactly *one* EC2 server and *one* RDS database running 24/7 (750 hours/month). If the engineering team provisions both the Staging and Production servers in AWS simultaneously, the account will exceed the free tier limit and incur charges (approx. \$20-\$30/month). **Recommendation:** Run the Staging database and server locally on developer machines using Docker Compose, and reserve the AWS Free Tier exclusively for the Production environment.



2. Sprint 1: Cloud Foundation & IAM Security

Objective: Secure the AWS root account, configure proactive billing alarms to prevent surprise charges, and create an isolated, strictly-scoped IAM Developer account.

Step 2.1: AWS Budgets & Billing Alarms (Zero Spend)

This step must be performed by the Founder/CTO using the Root AWS Account.

1. Log in to the AWS Management Console as the Root User.
2. In the top navigation search bar, type **AWS Budgets** and select it.
3. Click the orange **Create budget** button on the right side.
4. Under **Budget setup**, select **Use a template (simplified)**.
5. Under **Templates**, select **Zero spend budget**. *(This triggers an alert the second your bill hits \$0.01, ensuring the Free Tier is actually working).*
6. Under **Email recipients**, type in the primary founder/engineering email addresses (comma separated).
7. Click **Create budget**.

Step 2.2: The Developer IAM Policy

We will now create a strict permissions boundary for the engineering team.

1. In the top search bar, type **IAM** and open the Identity and Access Management console.
2. In the left-hand navigation pane, click **Policies**, then click the orange **Create policy** button.
3. **CRITICAL:** Do NOT use the default "Visual" editor. Click the **JSON** tab in the top right.
4. Delete all the default code in the editor, and paste the following payload exactly:

```

{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": [
        "ec2:*",
        "rds:*",
        "s3:*",
        "cognito-idp:*",
        "ecr:*",
        "iam:CreateServiceLinkedRole"
      ],
      "Resource": "*"
    },
    {
      "Effect": "Allow",
      "Action": "iam:PassRole",
      "Resource": "*",
      "Condition": {
        "StringEquals": {
          "iam:PassedToService": [
            "rds.amazonaws.com",
            "ec2.amazonaws.com"
          ]
        }
      }
    },
    {
      "Effect": "Deny",
      "Action": [
        "aws-portal:*",
        "iam:CreateUser",
        "iam:DeleteUser"
      ],
      "Resource": "*"
    }
  ]
}

```

5. Click **Next** at the bottom right.
6. Name the policy: `Stacy-Developer-Policy`.
7. Scroll down and click **Create policy**.

Step 2.3: The Developer IAM User

1. Still in the IAM console, click **Users** in the left navigation pane.

2. Click **Create user**.

3. **User details:**

- User name: `staye-developer`
- Check the box: **Provide user access to the AWS Management Console**.
- Select **I want to create an IAM user**.
- Select **Custom password**, type a secure password, and uncheck "Users must create a new password at next sign-in".
- Click **Next**.

4. **Permissions:**

- Select **Attach policies directly**.
- In the search bar, search for `Staye-Developer-Policy`. Check the box next to it.
- Click **Next**, then **Create user**.
- Note down the Console sign-in URL provided on the success screen.

5. **Generate CLI Access Keys:**

- Click on the `staye-developer` user you just created.
- Go to the **Security credentials** tab.
- Scroll down to **Access keys** and click **Create access key**.
- Select **Command Line Interface (CLI)**, check the confirmation box, and click **Next**.
- Click **Create access key**.
- **CRITICAL:** Copy the `Access key ID` and the `Secret access key` to your `.env` file immediately. Once you leave this screen, the Secret Key is gone forever.

6. **Log out of the Root account immediately.** For all subsequent sprints, log in using the `staye-developer` credentials via the custom sign-in URL.

3. Sprint 2: Network Security & Firewalls

Objective: Create isolated VPC Security Groups (Firewalls) to ensure the database can only be accessed by the Next.js server, locking out the public internet.

Ensure you are logged in as `staye-developer` in the `ap-south-1` (Mumbai) region.

Step 3.1: Create the Web Server Firewall (`staye-web-sg`)

1. In the top search bar, type **EC2** and open the EC2 Dashboard.
2. In the left navigation pane, scroll down to the **Network & Security** section and click **Security Groups**.
3. Click the orange **Create security group** button.
4. **Basic details:**
 - Security group name: `staye-web-sg` (Do not use `sg-` as it is a reserved prefix).
 - Description: `Allows HTTP/HTTPS from public, SSH from Developer IP`.
 - VPC: Leave as the default VPC.
5. **Inbound rules:** Click **Add rule** three times to create three rules:
 - **Rule 1 (HTTP):** Type: `HTTP`, Source: `Anywhere-IPv4` (`0.0.0.0/0`).
 - **Rule 2 (HTTPS):** Type: `HTTPS`, Source: `Anywhere-IPv4` (`0.0.0.0/0`).
 - **Rule 3 (SSH):** Type: `SSH`, Source: `My IP` (AWS will auto-fill your current IP. If your IP changes later, you must update this rule to SSH into the server).
6. Scroll to the bottom and click **Create security group**.

Step 3.2: Create the Database Firewall (`staye-db-sg`)

1. You should still be on the Security Groups page. Click **Create security group** again.
2. **Basic details:**
 - Security group name: `staye-db-sg`
 - Description: `Allows database traffic ONLY from the staye-web-sg server`.
 - VPC: Leave as the default VPC.
3. **Inbound rules:** Click **Add rule** once:
 - **Rule 1 (Postgres):** Type: `PostgreSQL`.
 - **Source:** Click the search box next to Custom. Start typing `staye-web-sg`. A dropdown will appear showing the Security Group ID (e.g., `sg-0abc ...`). **You MUST click the dropdown suggestion** or AWS will throw a CIDR block error.
4. Scroll to the bottom and click **Create security group**.

4. Sprint 3: Exhaustive AWS Resource Provisioning

Objective: Spin up the exact Free Tier compatible infrastructure components required for the Next.js backend to function.

Step 4.1: Amazon RDS (PostgreSQL Database)

Warning: The RDS UI contains numerous hidden traps designed to upsell enterprise features. Follow these settings exactly.

1. In the top search bar, type **RDS** and click it.
2. Click the orange **Create database** button.
3. **Database creation method:** Select **Standard create**.
4. **Engine options:** Select **PostgreSQL**. Leave the version on the default stable release (e.g., PostgreSQL 16.x).
5. **Templates:** Select **Dev/Test** or **Sandbox**. *(Do NOT select Production. If forced to select Production, you must manually override all settings below).*
6. **Availability and durability: CRITICAL:** Explicitly select **Single-AZ DB instance deployment (1 instance)** on the far right.
7. **Credentials Settings:**
 - Master username: `postgres`
 - **Credentials management:** Select **Self managed**. *(Do NOT use AWS Secrets Manager, it costs \$0.40/month).*
 - Master password: Type a secure 12+ character password. **Do not use** `@`, `/`, `'`, or `"` characters, **as they break the PostgreSQL connection string**.
8. **Instance configuration:**
 - DB instance class: Select **Burstable classes (includes t classes)**.
 - Instance type: Open the dropdown and select `db.t3.micro` or `db.t4g.micro`. *(This is the free tier instance).*
9. **Storage:**
 - Storage type: Select **General Purpose SSD (gp2)**. *(Do NOT use io2 or gp3 unless necessary, io2 costs hundreds of dollars).*
 - Allocated storage: **20 GiB**.
 - **Storage autoscaling:** **UNCHECK** "Enable storage autoscaling".
10. **Connectivity:**
 - Compute resource: **Don't connect to an EC2 compute resource**.

- VPC: Default VPC.
- Public access: Select **No**.
- VPC security group: Click the 'X' to remove `default`. Select `staye-db-sg`.

11. Additional configuration (Open the dropdown at the very bottom):

- Initial database name: Type `staye_db`.
- **Backup: UNCHECK** "Enable automated backup". (20GB of data + 7 days of backups exceeds the 20GB free tier limit and throws a retention error).
- **Encryption: UNCHECK** "Enable encryption". (Prevents the `kms:ListAliases` error).
- **Monitoring: UNCHECK** "Enable Performance Insights" and **UNCHECK** "Enable Enhanced monitoring". (These cost money and throw `iam:ListRoles` errors).

12. Review the "Estimated monthly costs" box at the bottom. It should confirm you are covered by the Free Tier. Click **Create database.**

Step 4.2: Amazon S3 (Document Storage Bucket)

1. In the top search bar, type **S3** and click it.

2. Click **Create bucket**.

3. General configuration:

- Bucket name: `staye-production-documents-unique-id` (Must be globally unique. Add a suffix like `-v1` if taken).
- AWS Region: `ap-south-1 (Mumbai)`.

4. Block Public Access settings:

- **CHECK** the box for **Block all public access**. (Files will be served securely via presigned URLs from Next.js).

5. Click **Create bucket**.

6. CORS Configuration (Required for Next.js uploads):

- Click on your newly created bucket.
- Go to the **Permissions** tab.
- Scroll down to **Cross-origin resource sharing (CORS)** and click **Edit**.
- Paste the following JSON:

```
[
  {
    "AllowedHeaders": ["*"],
    "AllowedMethods": ["GET", "PUT", "POST", "DELETE"],
    "AllowedOrigins": ["*"],
    "ExposeHeaders": []
  }
]
```

- Click **Save changes**. (Note: For production, change `AllowedOrigins` from `*` to `https://app.staye.com`).

Step 4.3: Amazon Cognito (Authentication User Pool)

1. In the top search bar, type **Cognito** and click it.
2. Click **Create user pool**.
3. **Step 1: Sign-in experience:** Select **Email**. Click Next.
4. **Step 2: Security requirements:** Leave Password defaults. Select **No MFA**. Click Next.
5. **Step 3: Sign-up experience:** **UNCHECK** "Enable self-registration". (Tenants are invited, they do not sign up). Click Next.
6. **Step 4: Message delivery:** Select **Send email with Cognito** (Free tier covers 50 emails/day). Click Next.
7. **Step 5: App integration:**
 - User pool name: `staye-user-pool`
 - Initial app client: Select **Confidential client**.
 - App client name: `staye-nextjs-client`
 - Generate a client secret: **Use default settings** (Generate a client secret).
 - Under Authentication flows, ensure **ALLOW_USER_PASSWORD_AUTH** is checked so the backend can manually authenticate users. Click Next.
8. **Step 6:** Review and click **Create user pool**.

Step 4.4: Amazon ECR (Elastic Container Registry)

1. In the top search bar, type **ECR** and click **Elastic Container Registry**.
 2. Click **Create repository**.
 3. Visibility settings: **Private**.
 4. Repository name: `staye-backend`.
 5. Image scan settings: **CHECK** "Scan on push".
 6. Click **Create repository**.
-

5. Sprint 4: Codebase Refactoring (Moving off Supabase)

Objective: Rip out all Supabase SDK logic from the Next.js API routes and wire the application directly to the AWS APIs.

Step 5.1: Prisma Database Migration

1. Update the `.env` file with the new RDS connection string:

```
# Format: postgresql://[USER]:[PASSWORD]@[RDS_ENDPOINT]:5432/[DB_NAME]
DATABASE_URL="postgresql://postgres:YourSecurePassword@staye-db.cwxyz.ap-south-1.rds.amazonaws.com:5432/staye_db"
```

2. Run Prisma migrations against the new database:

```
npx prisma migrate deploy
npx prisma generate
```

Step 5.2: AWS SDK S3 Refactoring

1. Uninstall Supabase storage dependencies.
2. Install the AWS SDK:

```
npm install @aws-sdk/client-s3 @aws-sdk/s3-request-presigner
```

3. Create a utility file `lib/s3Client.ts`:

```
import { S3Client } from "@aws-sdk/client-s3";

export const s3 = new S3Client({
  region: process.env.AWS_REGION,
  credentials: {
    accessKeyId: process.env.AWS_ACCESS_KEY_ID!,
    secretAccessKey: process.env.AWS_SECRET_ACCESS_KEY!,
  }
});
```

4. Refactor Upload API Routes (e.g., `app/api/upload/route.ts`):

- Replace `supabase.storage.from('kyc').upload()` with `PutObjectCommand`.

```
import { PutObjectCommand } from "@aws-sdk/client-s3";
import { s3 } from "@lib/s3Client";

// Inside route handler...
const command = new PutObjectCommand({
  Bucket: process.env.AWS_S3_BUCKET_NAME,
  Key: `kyc/${tenantId}-${filename}`,
  Body: fileBuffer,
  ContentType: mimeType,
});
await s3.send(command);
```

Step 5.3: Authentication Refactoring (Cognito)

1. Uninstall `@supabase/supabase-js` and `@supabase/ssr`.
2. Depending on architecture, implement AWS Cognito. The recommended approach for Next.js App Router is using `NextAuth.js` (Auth.js) with the Cognito Provider.
3. Install NextAuth:

```
npm install next-auth
```

4. Configure `app/api/auth/[...nextauth]/route.ts`:

```
import NextAuth from "next-auth"
import CognitoProvider from "next-auth/providers/cognito"

export const authOptions = {
  providers: [
    CognitoProvider({
      clientId: process.env.COGNITO_CLIENT_ID,
      clientSecret: process.env.COGNITO_CLIENT_SECRET,
      issuer: process.env.COGNITO_ISSUER,
    })
  ]
}

const handler = NextAuth(authOptions)
export { handler as GET, handler as POST }
```

5. Update `middleware.ts` to use `withAuth` from NextAuth instead of the Supabase session checker, redirecting unauthenticated users to the login route.

6. Sprint 5: Dockerization & Automated CI/CD Pipeline

Objective: Containerize the Next.js monolithic application and fully automate the deployment pipeline to EC2 using GitHub Actions.

Step 6.1: The Production Dockerfile

Create `Dockerfile` in the root directory. This multi-stage build significantly reduces the final image size.

```
# Base node image
FROM node:18-alpine AS base
WORKDIR /app
COPY package.json package-lock.json ./
RUN npm ci

# Build stage
FROM base AS builder
COPY . .
# Generate prisma client for linux
RUN npx prisma generate
RUN npm run build

# Production stage
FROM node:18-alpine AS runner
WORKDIR /app
COPY --from=builder /app/package.json ./
COPY --from=builder /app/.next/standalone ./
COPY --from=builder /app/.next/static ./next/static
COPY --from=builder /app/public ./public

# Set production env
ENV NODE_ENV=production
ENV PORT=3000

EXPOSE 3000
CMD ["node", "server.js"]
```

Required: Open `next.config.js` and add `output: 'standalone'` to the config object.

Step 6.2: Configure AWS Systems Manager (SSM)

Instead of paying for AWS Secrets Manager, we will store environment variables securely for free.

1. Go to **AWS Systems Manager** → **Parameter Store**.
2. Click **Create parameter**.
3. Create a parameter named `/stage/prod/DATABASE_URL`. Set the type to **SecureString** and paste the RDS connection string.
4. Repeat for `COGNITO_CLIENT_ID`, `AWS_ACCESS_KEY_ID`, etc.

Step 6.3: GitHub Actions Pipeline

Create `.github/workflows/deploy.yml`. When code is merged to `main`, this pipeline builds the Docker image, pushes it to ECR, and tells the EC2 instance to restart.

```

name: Deploy to AWS EC2
on:
  push:
    branches: [ "main" ]

jobs:
  deploy:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3

      - name: Configure AWS credentials
        uses: aws-actions/configure-aws-credentials@v2
        with:
          aws-access-key-id: ${ secrets.AWS_ACCESS_KEY_ID }
          aws-secret-access-key: ${ secrets.AWS_SECRET_ACCESS_KEY }
          aws-region: ap-south-1

      - name: Login to Amazon ECR
        id: login-ecr
        uses: aws-actions/amazon-ecr-login@v1

      - name: Build, tag, and push image to Amazon ECR
        env:
          ECR_REGISTRY: ${ steps.login-ecr.outputs.registry }
          ECR_REPOSITORY: staye-backend
          IMAGE_TAG: latest
        run: |
          docker build -t $ECR_REGISTRY/$ECR_REPOSITORY:$IMAGE_TAG .
          docker push $ECR_REGISTRY/$ECR_REPOSITORY:$IMAGE_TAG

      - name: Deploy to EC2 via AWS SSM
        run: |
          aws ssm send-command \
            --instance-ids "${ secrets.EC2_INSTANCE_ID }" \
            --document-name "AWS-RunShellScript" \
            --parameters commands=[ \
              "aws ecr get-login-password --region ap-south-1 | docker login --username AWS --
password-stdin ${ steps.login-ecr.outputs.registry }", \
              "docker pull ${ steps.login-ecr.outputs.registry }/staye-backend:latest", \
              "docker stop staye-api || true", \
              "docker rm staye-api || true", \
              "export DATABASE_URL=$(aws ssm get-parameter --name /staye/prod/DATABASE_URL --with-
decryption --query Parameter.Value --output text)", \
              "docker run -d --name staye-api -p 80:3000 -e DATABASE_URL=$DATABASE_URL ${ steps.login-ecr.outputs.registry }/staye-backend:latest" \
            ]

```


7. Sprint 6: Data Cutover & Go-Live

Objective: Safely migrate existing production data from Supabase to AWS with zero data loss, flip the DNS records, and formally Go-Live.

Step 7.1: The Maintenance Freeze

1. Schedule a 2-hour maintenance window during off-peak hours (e.g., 2:00 AM IST).
2. Deploy a maintenance banner to the frontend to prevent users from modifying data.
3. In the Supabase Dashboard, temporarily revoke the API keys to guarantee no new writes occur during the copy process.

Step 7.2: Database Migration (`pg_dump`)

1. **Export from Supabase:** Use `pg_dump` from your terminal to extract the entire database schema and data.

```
pg_dump -U postgres -h db.supabase.co -F c -d postgres > stayer_backup.dump
```

2. **Import to AWS RDS:** Use `pg_restore` to push the data into the new RDS instance.

```
pg_restore -U postgres -h stayer-db.cwxyz.ap-south-1.rds.amazonaws.com -d stayer_db < stayer_backup.dump
```

Step 7.3: Storage Migration (S3)

1. Write a temporary Node.js script using the Supabase SDK to list and download all existing KYC documents and PDFs to your local machine.
2. Extend the script using the AWS SDK `PutObjectCommand` to iterate through the local files and upload them directly into the new AWS S3 `stayer-production-documents` bucket. Ensure you maintain the exact same folder structure (e.g., `kyc/tenant-123-id.jpg`).

Step 7.4: Auth User Cutover

Warning: Passwords cannot be extracted from Supabase due to bcrypt hashing security.

1. Export the list of user emails and names from Supabase Auth as a CSV.

2. Write a Node.js script utilizing the Cognito `adminCreateUser` API to create these accounts in AWS Cognito.
3. Pass the `MessageAction: 'SUPPRESS'` flag if you do not want Cognito to automatically email them immediately.
4. **Tenant Communication:** Send an automated email/SMS to all existing users using your current mailing provider: *"Staye has upgraded our systems. Please click here to set a new password for your account."*

Step 7.5: DNS Flip (Go-Live)

1. Log into your domain registrar (e.g., GoDaddy, Namecheap) or Route 53.
2. Update the `A` record for `app.staye.com` to point to the Elastic IP address of your AWS EC2 instance.
3. Once DNS propagates (usually 10-30 minutes), verify the platform is running on the AWS infrastructure by logging in as an admin.
4. Go back to Supabase, disable the project to prevent accidental usage, and stop your billing.

End of Runbook V3